Hey! I'd be happy to help break down this expression for you.

You're dealing with a lambda expression and the definition of a function someTerm that has a complex type. The goal is to understand how the left part (the type and function signature) is transformed into the right part (the actual lambda term implementation).

Let's step through this:

1. The function signature (left part)

You have:

Definition someTerm (A P Q : Ty) : Tm ((A ⇒ Q) ⇒ ((((P ⇒ Q) ⇒ P) ⇒ P) ⇒ A) ⇒ A) := ...

Type breakdown:

- A, P, Q: These are type variables.
- Ty: A kind that represents the set of all types.
- Tm: A term of a certain type.
- ⇒: This is the function type constructor.

So, let's break down the function type ((A ⇒ Q) ⇒ ((((P ⇒ Q) ⇒ P) ⇒ P) ⇒ A) ⇒ A):

- The type of someTerm is a function that:
    - Takes an argument of type (A ⇒ Q) (a function from A to Q).
    - Then, it takes an argument of type ((((P ⇒ Q) ⇒ P) ⇒ P) ⇒ A), which is itself a function.
    - Finally, it returns a value of type A.

To simplify, let's rewrite the type in more digestible steps:

1. A ⇒ Q: A function from A to Q.
2. P ⇒ Q: A function from P to Q.
3. (P ⇒ Q) ⇒ P: A function that takes a function of type P ⇒ Q and returns a P.
4. ((P ⇒ Q) ⇒ P) ⇒ P: A function that takes a function of type (P ⇒ Q) ⇒ P and returns a P.
5. (((P ⇒ Q) ⇒ P) ⇒ P) ⇒ A: A function that takes a function of type ((P ⇒ Q) ⇒ P) ⇒ P and returns A.

So, the final type of someTerm is a function that:

- Takes a function (A ⇒ Q) as its first argument.

- Takes a function (((P ⇒ Q) ⇒ P) ⇒ P) ⇒ A as its second argument.
- And returns a value of type A.

2. The lambda term (right part)

Now, the right-hand side of the definition is:

lam (fun x => lam (fun y => y · (lam (fun z => z · (lam (fun w => x · (y · lam (fun v => w))))))))

This is the body of the function someTerm, and it's a chain of lambda expressions. Let's go step-by-step through the term:

1st Layer: lam (fun x => ...)

The outermost lambda expression defines the first argument of the function. x is of type (A ⇒ Q). This means x is a function that takes an argument of type A and returns something of type Q.

2nd Layer: lam (fun y => ...)

The second lambda defines the second argument, y, which is of type (((P ⇒ Q) ⇒ P) ⇒ P) ⇒ A. This is a function that takes a function of type ((P ⇒ Q) ⇒ P) ⇒ P and returns a value of type A.

3rd Layer: y · (lam (fun z => ...))

Inside the second layer, the body of the second lambda is y · (lam (fun z => ...)). Here, · is presumably some application operation, meaning that y is being applied to the result of the following lambda expression.

The lam (fun z => ...) defines z, which is an argument of type (P ⇒ Q) ⇒ P.

4th Layer: z · (lam (fun w => ...))

In the body of the third lambda expression, z is applied to the result of the next lambda expression. This lambda defines w, which is of type P.

5th Layer: x · (y · lam (fun v => w))

Now, we get to the final bit: x · (y · lam (fun v => w)).

- y · lam (fun v => w) applies the function y to the lambda lam (fun v => w), which defines v of type P.
- Finally, x is applied to the result of that, and x is a function from A to Q.

The term as a whole is a chain of applications that ultimately evaluates to a term of type A.

How does the left side transform into the right side?

To map the left part (the type signature) to the right part (the lambda term), we use the following principles of functional programming and lambda calculus:

1. **Currying**: The function is curried, meaning it takes one argument at a time.
2. **Function application**: Each lambda term is an application of a function to an argument.
3. **Type matching**: The types of the lambda terms and their arguments are designed to match the type signature. For example, the term x is a function of type A ⇒ Q, and y is a function of type (((P ⇒ Q) ⇒ P) ⇒ P) ⇒ A.

Each lambda expression creates a new function that fits the type of the argument it takes and applies it to other functions in the chain. The term x · (y · lam (fun v => w)) represents the "final" result, which is a value of type A.

This is a pretty detailed breakdown, but the general idea is that the function is carefully constructed to take two arguments—one of type A ⇒ Q and one of a more complex functional type—and then it applies these arguments in a nested way to eventually return a result of type A.

Would you like me to go deeper into any particular part of the term, or do you want to see a visual derivation tree for better clarity?